



Curso:

Disciplina: Programação de Sistemas Web

Professor: Lucas de Oliveira Teixeira

Aluno: _____

Data: _____

R.A.: _____

Trabalho - Parte 2 - Aplicação Full Stack

1. Introdução

Esta especificação descreve a **Parte 2** do projeto integrador da disciplina de **Programação de Sistemas Web**. Ela dá continuidade à Parte 1: o mesmo aplicativo **CineTrack** evolui de um frontend que roda apenas no navegador para uma aplicação **full stack**.

Assume-se que a Parte 1 já foi concluída. O sistema, o modelo de dados (título, ano, gênero, pôster, nota, status e comentário) e o conjunto de 6 filmes de exemplo são os mesmos definidos na Parte 1 e não são repetidos aqui. As regras gerais do trabalho (formação de grupos, autoria e plágio, formato do repositório) também seguem as da Parte 1.

Nesta etapa, a interface é **reescrita em React** com componentes reutilizáveis, e um servidor **Node.js** com o `mini-express.js/express.js` expõe uma API REST. Os dados passam a ser persistidos em um arquivo JSON no servidor. A pasta `parte1/` permanece no repositório exatamente como na tag `v1.0`. A versão final desta parte é marcada pela tag `v2.0`.

Como na Parte 1, esta especificação descreve o comportamento esperado, não uma implementação única: diferentes organizações de código, nomes e estratégias são aceitos, desde que os requisitos sejam atendidos.

2. Requisitos

Tecnologias: React (Vite) · Node.js · mini-Express · JSON (persistência)

O mesmo sistema desenvolvido na Parte 1 evolui para uma aplicação full stack. A interface React conversa com uma API REST servida por Node.js, e os dados ficam em um arquivo JSON no servidor.

2.1 Requisitos Funcionais

Os requisitos abaixo descrevem **o que** a aplicação deve fazer. A forma de implementar cada um fica a critério do aluno.

2.1.1 RF-01: Interface em componentes React

O frontend deve ser reescrito em React, organizado em **componentes reutilizáveis** com responsabilidades bem definidas. Espera-se, no mínimo, componentes para: o cabeçalho com busca e o botão de adicionar; os filtros por status; a lista (grade) de filmes; o card de um filme; e o formulário de adição/edição. Toda atualização de estado deve ser feita **sem mutação**, e a remoção deve pedir confirmação, como na Parte 1.

2.1.2 RF-02: Acesso a dados no frontend

A comunicação com o servidor deve ficar **concentrada em um único módulo** de acesso a dados, com operações para listar, criar, atualizar e remover filmes. A aplicação carrega a lista ao iniciar e trata no render os estados de **carregando**, **erro** e **vazio**. As operações de escrita devem aguardar a resposta do servidor antes de atualizar a interface (uso de `async/await`).

2.1.3 RF-03: API REST com Node.js + mini-Express/Express

O servidor deve expor uma API REST com os seguintes *endpoints*:



Método	Rota	Descrição
GET	/filmes	Retorna a lista completa de filmes em JSON; aceita o filtro ?status=.
GET	/filmes/:id	Retorna o filme com o id fornecido, ou 404 se não encontrado.
POST	/filmes	Recebe um novo filme no corpo (JSON); responde 400 se faltar título ou ano; atribui um id único, salva e retorna o filme criado com status 201.
PUT	/filmes/:id	Atualiza o filme, preservando o id original. Retorna o filme atualizado ou 404 se não encontrado.
DELETE	/filmes/:id	Remove o filme. Retorna 204 em caso de sucesso ou 404 se não encontrado.

As respostas devem sair com Content-Type: application/json, e o servidor deve permitir as requisições do frontend em desenvolvimento (localhost), habilitando CORS.

2.1.4 RF-04: Persistência em arquivo JSON

Os filmes devem ser lidos e gravados em um arquivo JSON no servidor, usando os módulos nativos do Node. Toda operação de escrita (POST, PUT, DELETE) atualiza o arquivo, de modo que os dados sobrevivam ao reinício do servidor. Deve haver uma forma de popular o arquivo com os 6 filmes de exemplo da Parte 1 (um script de seed). Não é necessário banco de dados.

2.1.5 RF-05: Tratamento de erros no frontend

O frontend deve tratar falhas de comunicação com o servidor: os estados carregando, erro e vazio aparecem no render, e uma falha de requisição (rede indisponível, resposta 4xx/5xx) exibe mensagem visível ao usuário, sem travar a interface.

2.1.6 RF-06: Manutenção das funcionalidades da Parte 1

Todas as funcionalidades da Parte 1 (busca, filtro por status, adição, edição, remoção com confirmação, validação de formulário) devem continuar presentes e funcionais na versão React.

2.1.7 RF-07: Proxy TMDB (opcional)

Item **opcional**: uma rota no backend que consulta a API do TMDB com a chave lida de uma variável de ambiente. A chave fica em um arquivo de ambiente e nunca chega ao navegador.

2.2 Requisitos Não Funcionais

- **Organização**: frontend e backend em pastas separadas (frontend/ e backend/); parte1/ preservada.
- **Variáveis de ambiente**: configurações sensíveis (como a porta e chaves de API) ficam em um arquivo de ambiente fora do git; um arquivo de exemplo, sem valores reais, é versionado.
- **Ausência de dados sensíveis**: nenhuma senha, token ou chave de API no repositório ou no código do frontend.

2.3 Estrutura de Arquivos Sugerida

A estrutura abaixo é uma sugestão. O essencial é que parte1/ seja preservada e que o frontend React e o backend Node fiquem em pastas separadas.

```
cinetrack/  
|-- README.md  
|-- .gitignore
```



```
|-- docs/
|-- parte1/
|-- backend/
|   |-- server.js
|   |-- mini-express.js
|   |-- seed.js
|   |-- package.json
|   |-- .env.example
|   '-- data/
|       '-- filmes.json
'-- frontend/
    |-- index.html
    |-- package.json
    '-- src/
        |-- main.jsx
        |-- App.jsx
        '-- components/
```

Listing 1: Estrutura sugerida na tag v2.0 (Parte 2)

O arquivo de ambiente do backend existe localmente, mas não aparece no repositório. Demais arquivos gerados pelo Vite são mantidos como a ferramenta os criou.

2.4 O que será avaliado

A avaliação considera, de forma qualitativa:

- Organização da interface em componentes React com responsabilidades claras e estado sem mutação;
- API REST funcional com o `mini-express.js/express.js` e status HTTP adequados;
- Persistência em arquivo JSON, com dados preservados ao reiniciar o servidor;
- Integração frontend-backend, com carregamento inicial e escrita após a resposta do servidor;
- Tratamento dos estados de carregando, erro e vazio;
- Manutenção das funcionalidades da Parte 1 na versão React;
- Organização das pastas, README, ausência de segredos no repositório e histórico de *commits*.

2.5 Entrega

Formato de entrega: link do mesmo repositório Git utilizado na Parte 1, com a tag v2.0 criada.

O que deve ser entregue:

- Código-fonte completo: `parte1/` preservada, `frontend/` React e `backend/` com o mini-Express/Express;
- Arquivo `README.md` com: nome(s) do(s) aluno(s), RA(s) e as instruções para rodar o backend e o frontend;
- Arquivo de exemplo de variáveis de ambiente do backend (sem valores reais).

Bom trabalho e bons estudos!